

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #7: Bidireccionalidad (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 17 de
noviembre de 2024

Contenido

OBJETIVO	3
DIFICULTADES ENCONTRADAS	3
CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)	4
CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)	12
Conclusiones	14

OBJETIVO

Actualizar el proyecto agregando bidireccionalidad, es decir, que todos puedan funcionar como servidor y cliente, esto en caso de que el actual servidor empiece a utilizar más sus recursos, por tanto, va a bajar su ranking y entonces deberá realizarse un switch, lanzando como servidor el cliente con mayor ranking.

DIFICULTADES ENCONTRADAS

Nuestro único y principal problema fue con la actualización de la bidireccionalidad, si bien lo configurábamos, o salíamos todos como Servidor o no se modificaba en nada. Primero lo intentamos con una función extra pero no funcionó, tampoco por separado ni como una variable global. Al final, logramos solucionarlo usando la función dentro del Ranking, que en este caso es quien tiene el mayor CPU libre.

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)

SERVIDOR

```
1 package Ej;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import javax.swing.table.TableRowSorter;
6
7 import java.awt.*;
8 import java.io.*;
9 import java.net.*;
10 import java.util.*;
11 import java.util.concurrent.*;
12 import java.util.List;
13 import java.io.File;
14
15 import oshi.SystemInfo;
16 import oshi.hardware.CentralProcessor;
17 import oshi.hardware.GlobalMemory;
18 import oshi.hardware.HardwareAbstractionLayer;
19 import oshi.hardware.HWDiskStore;
20 import oshi.hardware.NetworkIF;
21 import oshi.software.os.OperatingSystem;
22
23 public class Server2 {
24     private static final int UPDATE_INTERVAL = 5000; // 5 segundos
25     private static Map<String, ClientHandler> connectedClients = new ConcurrentHashMap<>();
26     private static DefaultTableModel model;
27     private static JTable table;
28     private static final int SERVER_PORT = 8080; // Puerto estático
29
30     //COLORES
31     private static final String RESET = "\033[0m"; //Restablecer el color
32     private static final String RED = "\033[31m"; //R
33     private static final String GREEN = "\033[32m"; //V
34     private static final String YELLOW = "\033[33m"; //A
35     private static final String BLUE = "\033[34m"; //AZ
36
37
38     public static void main(String[] args) {
39         String[] columnNames = {
40             "ID", "Cliente/Servidor", "Modelo Procesador", "Velocidad (GHz)",
41             "Núcleos", "Disco Duro Total (GB)", "SO", "CPU Libre (%)",
42             "Memoria Libre (GB)", "Ancho Banda Libre (%)", "Disco Libre (GB)", "Estado"
43         };
44
45         model = new DefaultTableModel(columnNames, 0) {
46             @Override
47             public boolean isCellEditable(int row, int column) {
48                 return false;
49             }
50         };
51         table = new JTable(model);
52
53         // Configurar el frame
54         JFrame frame = new JFrame("Servidor");
55         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
56         frame.add(new JScrollPane(table));
57         frame.setSize(1200, 400);
58         frame.setVisible(true);
59
60         System.out.println(GREEN + "Esperando conexiones..." + RESET);
61
62         // Configurar el servidor con un puerto estático
63         try (ServerSocket serverSocket = new ServerSocket(SERVER_PORT)) {
64             // Agregar datos del servidor
65             addServerData();
66
67             // Hilo para actualizar datos del servidor y ordenar filas
68             ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
69             scheduler.scheduleAtFixedRate(() -> {
70                 updateServerData();
71                 updateClientsData();
72                 sortTableByCpuFree();
73             }, 0, UPDATE_INTERVAL, TimeUnit.MILLISECONDS);
74         }
```

```

74
75 // Aceptar conexiones de clientes
76 while (true) {
77     Socket clientSocket = serverSocket.accept();
78     ClientHandler handler = new ClientHandler(clientSocket);
79     new Thread(handler).start();
80 }
81 } catch (Exception e) {
82     e.printStackTrace();
83 }
84 }
85
86 private static void addServerData() {
87     SystemInfo si = new SystemInfo();
88     HardwareAbstractionLayer hal = si.getHardware();
89     CentralProcessor processor = hal.getProcessor();
90     OperatingSystem os = si.getOperatingSystem();
91     List<HWDiskStore> diskStores = hal.getDiskStores();
92
93     String processorModel = processor.getProcessorIdentifier().getName();
94     double processorSpeed = processor.getProcessorIdentifier().getVendorFreq() / 1e9;
95     int cores = processor.getLogicalProcessorCount();
96     long diskSize = diskStores.get(0).getSize() / (1024 * 1024 * 1024);
97
98
99     model.addRow(new Object[] {
100         System.getProperty("user.name"),
101         "SERVIDOR",
102         processorModel,
103         processorSpeed,
104         cores,
105         diskSize,
106         os.toString(),
107         0.0 // CPU libre
108         0.0, // Memoria Libre
109         0.0, // Ancho de banda
110         0.0, // Disco libre
111         "CONECTADO"
112     });
113 }
114
115 private static double updateServerData() {
116     SystemInfo si = new SystemInfo();
117     HardwareAbstractionLayer hal = si.getHardware();
118     CentralProcessor processor = hal.getProcessor();
119     GlobalMemory memory = hal.getMemory();
120     List<HWDiskStore> diskStores = hal.getDiskStores();
121
122     // Calcular CPU libre
123     long[] prevTicks = processor.getSystemCpuLoadTicks();
124     try { Thread.sleep(1000); } catch (InterruptedException e) { }
125     double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
126
127
128     // Calcular memoria libre
129     double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
130
131
132     // Calcular ancho de banda libre (aproximado basado en la utilización de red)
133     final double bandwidthFree;
134     List<NetworkIF> networks = hal.getNetworkIFs();
135     if (!networks.isEmpty()) {
136         NetworkIF net = networks.get(0);
137         net.updateAttributes();
138         long totalBytes = net.getBytesRecv() + net.getBytesSent();
139         bandwidthFree = 100.0 - (totalBytes % 100); // Simplificado para ejemplo
140     } else {
141         bandwidthFree = 100.0;
142     }
143 }

```

```

144 // Calcular espacio libre en disco
145 File rootDisk = new File("/");
146 double diskFree = rootDisk.getUsableSpace() / (1024.0 * 1024 * 1024); // Convertimos a GB
147
148 // Actualizar fila del servidor
149 SwingUtilities.invokeLater(() -> {
150     model.setValueAt(String.format("%.2f", cpuFree), 0, 7);
151     model.setValueAt(String.format("%.2f", freeMemory), 0, 8);
152     model.setValueAt(String.format("%.2f", bandwidthFree), 0, 9);
153     model.setValueAt(String.format("%.2f", diskFree), 0, 10);
154 });
155 return cpuFree;
156 }
157
158 private static void updateClientsData() {
159     // Crear una lista temporal de clientes a remover
160     List<String> clientsToRemove = new ArrayList<>();
161
162     // Verificar clientes desconectados
163     connectedClients.forEach((id, handler) -> {
164         if (!handler.isConnected()) {
165             updateClientStatus(id, "DESCONECTADO");
166             clientsToRemove.add(id);
167         }
168     });
169
170     // Remover clientes desconectados
171     clientsToRemove.forEach(connectedClients::remove);
172
173     // Actualizar la tabla solo para clientes conectados
174     for (int i = 0; i < model.getRowCount(); i++) {
175         String clientId = (String) model.getValueAt(i, 0);
176         String status = (String) model.getValueAt(i, 11);
177
178         // Si es el servidor, continuar con el siguiente
179         if (clientId.equals("SERVIDOR")) {
180             continue;
181         }
182
183         // Si el cliente está desconectado, mantener los valores pero en gris
184         if (status.equals("DESCONECTADO")) {
185             table.setRowSelectionInterval(i, i);
186             table.setSelectionBackground(Color.LIGHT_GRAY);
187         }
188     }
189 }
190
191 private static void updateClientStatus(String clientId, String status) {
192     for (int i = 0; i < model.getRowCount(); i++) {
193         if (model.getValueAt(i, 0).equals(clientId)) {
194             model.setValueAt(status, i, 11);
195
196             if (status.equals("DESCONECTADO")) {
197                 // Limpiar todos los campos excepto ID y tipo de cliente
198                 model.setValueAt("", i, 2); // Modelo Procesador
199                 model.setValueAt(0.0, i, 3); // Velocidad
200                 model.setValueAt(0, i, 4); // N°cleos
201                 model.setValueAt(0.0, i, 5); // Disco Duro Total
202                 model.setValueAt("", i, 6); // SO
203                 model.setValueAt(0.0, i, 7); // CPU Libre
204                 model.setValueAt(0.0, i, 8); // Memoria Libre
205                 model.setValueAt(0.0, i, 9); // Ancho Banda Libre
206                 model.setValueAt(0.0, i, 10); // Disco Libre
207
208                 // Cambiar color de fondo a gris
209                 table.setRowSelectionInterval(i, i);
210                 table.setSelectionBackground(Color.LIGHT_GRAY);
211             } else {
212                 table.clearSelection();
213                 table.setSelectionBackground(table.getBackground());

```

```

214         }
215         break;
216     }
217 }
218 sortTableByCpuFree();
219 }
220
221 private static void sortTableByCpuFree() {
222     TableRowSorter<DefaultTableModel> sorter = new TableRowSorter<>(model);
223
224     // Configurar comparador personalizado
225     sorter.setComparator(7, (value1, value2) -> {
226         int row1 = -1, row2 = -1;
227
228         // Encontrar las filas correspondientes a los valores
229         for (int i = 0; i < model.getRowCount(); i++) {
230             if (model.getValueAt(i, 7).equals(value1)) row1 = i;
231             if (model.getValueAt(i, 7).equals(value2)) row2 = i;
232         }
233
234         // Obtener estados de conexión
235         String status1 = model.getValueAt(row1, 11).toString();
236         String status2 = model.getValueAt(row2, 11).toString();
237
238         // Si uno está desconectado y el otro no, el desconectado va al final
239         if (status1.equals("DESCONECTADO") && !status2.equals("DESCONECTADO")) return 1;
240         if (!status1.equals("DESCONECTADO") && status2.equals("DESCONECTADO")) return -1;
241
242         // Si ambos están desconectados, mantener el orden actual
243         if (status1.equals("DESCONECTADO") && status2.equals("DESCONECTADO")) {
244             return row1 - row2;
245         }
246
247         // Asegurarse de que los valores sean válidos para convertir a Double
248         double cpu1 = parseDoubleSafely(value1);
249         double cpu2 = parseDoubleSafely(value2);
250
251         // Ordenar por CPU libre
252         return Double.compare(cpu2, cpu1); // Orden descendente
253     });
254
255     table.setRowSorter(sorter);
256
257     // Aplicar orden descendente en la columna 7 (CPU Libre)
258     List<RowSorter.SortKey> sortKeys = new ArrayList<>();
259     sortKeys.add(new RowSorter.SortKey(7, SortOrder.ASCENDING));
260     sorter.setSortKeys(sortKeys);
261     sorter.sort();
262
263     // Ahora, asignamos el tipo "Servidor" a quien tiene el CPU más alto
264     double maxCpuFree = -1;
265     int maxCpuRow = -1;
266
267     // Encontramos la fila con el CPU más alto
268     for (int i = 0; i < model.getRowCount(); i++) {
269         double cpuFree = parseDoubleSafely(model.getValueAt(i, 7)); // Valor de CPU Libre en la columna 7
270         if (cpuFree > maxCpuFree) {
271             maxCpuFree = cpuFree;
272             maxCpuRow = i;
273         }
274     }
275
276     // Asignamos "Servidor" a la fila con el CPU más alto y "Cliente" a las demás
277     for (int i = 0; i < model.getRowCount(); i++) {
278         if (i == maxCpuRow) {
279             model.setValueAt("SERVIDOR", i, 1); // Columna 1 es la que tiene el estado
280         } else {
281             model.setValueAt("CLIENTE", i, 1); // Asignamos "Cliente" a las demás filas
282         }
283     }
284 }
285

```

```

286 // Método seguro para convertir String a Double
287 private static double parseDoubleSafely(Object value) {
288     try {
289         return Double.parseDouble(value.toString());
290     } catch (NumberFormatException e) {
291         return 0.0; // Si el valor no es un número válido, se devuelve 0.0
292     }
293 }
294
295
296
297 static class ClientHandler implements Runnable {
298     private final Socket socket;
299     private String clientId;
300     private volatile boolean connected = true;
301     private ObjectInputStream ois;
302
303     public ClientHandler(Socket socket) {
304         this.socket = socket;
305     }
306
307     @Override
308     public void run() {
309         try {
310             ois = new ObjectInputStream(socket.getInputStream());
311             // Esperar hasta que se reciba el ID del cliente
312             Object receivedObject = ois.readObject();
313             if (receivedObject instanceof Map) {
314                 Map<String, Object> clientSpecs = (Map<String, Object>) receivedObject;
315                 clientId = (String) clientSpecs.get("ID");
316                 // Agregar al mapa de clientes conectados
317                 connectedClients.put(clientId, this);
318
319                 while (connected) {
320                     receivedObject = ois.readObject();
321                     if (receivedObject instanceof Map) {
322                         clientSpecs = (Map<String, Object>) receivedObject;
323                         handleClientData(clientSpecs);
324                     }
325                 }
326             }
327             } catch (EOFException e) {
328                 System.out.println(RED + "Cliente desconectado: " + RESET + clientId);
329                 connected = false;
330             } catch (IOException e) {
331                 System.out.println(RED + "Error de conexión con el cliente: " + RESET + clientId);
332                 connected = false;
333             } catch (ClassNotFoundException e) {
334                 e.printStackTrace();
335             } finally {
336                 try {
337                     if (ois != null) {
338                         ois.close();
339                     }
340                     socket.close();
341                 } catch (IOException e) {
342                     e.printStackTrace();
343                 }
344
345                 // Actualizar el estado del cliente en el servidor
346                 if (clientId != null) {
347                     connected = false;
348                     updateClientStatus(clientId, "DESCONECTADO");
349                 }
350             }
351         }
352
353     private void handleClientData(final Map<String, Object> clientSpecs) {
354         SwingUtilities.invokeLater(() -> {
355             boolean clientExists = false;
356             String clientId = (String) clientSpecs.get("ID");
357

```



```

358         for (int i = 0; i < model.getRowCount(); i++) {
359             if (model.getValueAt(i, 0).equals(clientId)) {
360                 // Si el cliente existe pero estaba desconectado, actualizar su estado
361                 if (model.getValueAt(i, 11).equals("DESCONECTADO")) {
362                     model.setValueAt("CONECTADO", i, 11);
363                     table.clearSelection();
364                     table.setSelectionBackground(table.getBackground());
365                 }
366                 updateRowData(i, clientSpecs);
367                 clientExists = true;
368                 break;
369             }
370         }
371
372         if (!clientExists) {
373             addNewClient(clientSpecs);
374         }
375
376         // Forzar reordenamiento después de actualizar
377         sortTableByCpuFree();
378     });
379 }
380
381 private void updateRowData(int row, Map<String, Object> specs) {
382     // Solo actualizar si el cliente está conectado
383     if (model.getValueAt(row, 11).equals("CONECTADO")) {
384         model.setValueAt(specs.get("Modelo del Procesador"), row, 2);
385         model.setValueAt(specs.get("Velocidad del Procesador (GHz)"), row, 3);
386         model.setValueAt(specs.get("Nucleos"), row, 4);
387         model.setValueAt(specs.get("Capacidad del Disco Duro (GB)"), row, 5);
388         model.setValueAt(specs.get("Version del Sistema Operativo"), row, 6);
389         model.setValueAt(specs.get("CPU Libre"), row, 7);
390         model.setValueAt(specs.get("Memoria Libre"), row, 8);
391         model.setValueAt(specs.get("Ancho de Banda Libre"), row, 9);
392         model.setValueAt(specs.get("Disco Libre"), row, 10);
393     }
394 }
395
396 private void addNewClient(Map<String, Object> specs) {
397     model.addRow(new Object[]{
398         specs.get("ID"),
399         "CLIENTE",
400         specs.get("Modelo del Procesador"),
401         specs.get("Velocidad del Procesador (GHz)"),
402         specs.get("Nucleos"),
403         specs.get("Capacidad del Disco Duro (GB)"),
404         specs.get("Version del Sistema Operativo"),
405         specs.get("CPU Libre"),
406         specs.get("Memoria Libre"),
407         specs.get("Ancho de Banda Libre"),
408         specs.get("Disco Libre"),
409         "CONECTADO"
410     });
411 }
412
413 public boolean isConnected() {
414     return connected;
415 }
416 }
417 }
418 }

```

CLIENTE

```
1 package Ej;
2
3 import oshi.SystemInfo;
4 import oshi.hardware.*;
5 import oshi.software.os.OperatingSystem;
6
7 import java.io.*;
8 import java.net.Socket;
9 import java.util.HashMap;
10 import java.util.List;
11 import java.util.Map;
12 import java.util.Scanner;
13
14 public class Client2 {
15     private static final int UPDATE_INTERVAL = 5000; // 5 seg
16     private static final String SERVER_IP = "25.51.170.2";
17     private static final int SERVER_PORT = 8080;
18
19     //COLORES
20     private static final String RESET = "\033[0m"; // Restablecer el color
21     private static final String RED = "\033[31m"; //Rojo
22     private static final String GREEN = "\033[32m"; //Verde
23     private static final String YELLOW = "\033[33m"; //Amarillo
24     private static final String BLUE = "\033[34m"; //Azul
25
26     public static void main(String[] args) {
27
28
29         try (Scanner scanner = new Scanner(System.in)) {
30             //ESTATICO
31             String serverIp = SERVER_IP;
32             int port = SERVER_PORT;
33
34             //ID
35             String clientId = getUserId();
36             Socket socket = new Socket(serverIp, port);
37             ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream());
38
39             //ACTUALIZACIONES
40             Thread updateThread = new Thread(() -> {
41                 try {
42                     while (!Thread.interrupted()) {
43                         Map<String, Object> specs = getSystemSpecs(clientId);
44                         oos.writeObject(specs);
45                         oos.reset(); // Limpiar cache
46                         Thread.sleep(UPDATE_INTERVAL);
47                     }
48                 } catch (Exception e) {
49                     System.out.println(RED + "Conexión perdida con el servidor.");
50                 }
51             });
52
53             updateThread.start();
54
55             //ESPERAR ENTER PARA DESCONECTAR
56             System.out.println(RED + "\nPresiona ENTER para desconectarte");
57             scanner.nextLine();
58             updateThread.interrupt();
59             socket.close();
60
61         } catch (Exception e) {
62             e.printStackTrace();
63         }
64     }
65
66     private static String getUserId() {
67         //Obtener el nombre del usuario actual del sistema
68         return System.getProperty("user.name");
69     }
70 }
```

```

72 private static Map<String, Object> getSystemSpecs(String clientId) {
73     SystemInfo si = new SystemInfo();
74     HardwareAbstractionLayer hal = si.getHardware();
75     CentralProcessor processor = hal.getProcessor();
76     GlobalMemory memory = hal.getMemory();
77     List<HwDiskStore> diskStores = hal.getDiskStores();
78     List<NetworkIF> networks = hal.getNetworkIFs();
79     OperatingSystem os = si.getOperatingSystem();
80
81     long[] prevTicks = processor.getSystemCpuLoadTicks();
82     try { Thread.sleep(1000); } catch (InterruptedException e) { }
83     double cpuFree = 100.0 - (processor.getSystemCpuLoadBetweenTicks(prevTicks) * 100);
84
85     double freeMemory = memory.getAvailable() / (1024.0 * 1024 * 1024);
86
87     double bandwidthFree = 100.0;
88     if (!networks.isEmpty()) {
89         NetworkIF net = networks.get(0);
90         net.updateAttributes();
91         long totalBytes = net.getBytesRecv() + net.getBytesSent();
92         bandwidthFree = 100.0 - (totalBytes % 100);
93     }
94
95     File rootDisk = new File("/");
96     double diskFree = rootDisk.getUsableSpace() / (1024.0 * 1024 * 1024);
97
98     Map<String, Object> specs = new HashMap<>();
99     specs.put("ID", clientId);
100    specs.put("Modelo del Procesador", processor.getProcessorIdentifier().getName());
101    specs.put("Velocidad del Procesador (GHz)", processor.getProcessorIdentifier().getVendorFreq() / 1e9);
102    specs.put("Nucleos", processor.getLogicalProcessorCount());
103    specs.put("Capacidad del Disco Duro (GB)", diskStores.get(0).getSize() / (1024 * 1024 * 1024));
104    specs.put("Version del Sistema Operativo", os.toString());
105    specs.put("CPU Libre", String.format("%.2f", cpuFree));
106    specs.put("Memoria Libre", String.format("%.2f", freeMemory));
107    specs.put("Ancho de Banda Libre", String.format("%.2f", bandwidthFree));
108    specs.put("Disco Libre", String.format("%.2f", diskFree));
109
110    System.out.println(GREEN + "\n----- ¡Datos Actualizandose! -----" + RESET);
111    System.out.println("ID: " + specs.get("ID"));
112    System.out.println(BLUE + "Modelo del Procesador: " + specs.get("Modelo del Procesador"));
113    System.out.println("Velocidad del Procesador: " + specs.get("Velocidad del Procesador (GHz)") + " GHz");
114    System.out.println("Nucleos: " + specs.get("Nucleos"));
115    System.out.println("Capacidad del Disco Duro: " + specs.get("Capacidad del Disco Duro (GB)") + " GB");
116    System.out.println("Version del Sistema Operativo: " + specs.get("Version del Sistema Operativo"));
117    System.out.println(YELLOW + "CPU Libre: " + specs.get("CPU Libre") + "%");
118    System.out.println("Memoria Libre: " + specs.get("Memoria Libre") + " GB");
119    System.out.println("Ancho de Banda Libre: " + specs.get("Ancho de Banda Libre") + "%");
120    System.out.println("Disco Libre: " + specs.get("Disco Libre") + " GB");
121
122
123    return specs;
124 }
125
126 }

```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)

SERVIDOR

Server.java
Server2.java
logback.xml
Dependencies

```
384         model.setValueAt(specs.get("Modelo d
385         model.setValueAt(specs.get("Velocida
386         model.setValueAt(specs.get("Nucleos"
387         model.setValueAt(specs.get("Capacida
388         model.setValueAt(specs.get("Version
389         model.setValueAt(specs.get("CPU Libr
390         model.setValueAt(specs.get("Memoria
391         model.setValueAt(specs.get("Ancho de
392         model.setValueAt(specs.get("Disco Li
393
394     }
```

Servidor

ID	Cliente/Servidor	Modelo Procesa	Velocidad (GHz)	Nucleos	Disco Duro Total	SO	CPU Libre (%)	Memoria Libre (...)	Ancho Banda Lib	Disco Libre (GB)	Estado
heidi	SERVIDOR	11th Gen Intel(R)	2.688	12	476	Microsoft Windo	97.32	1.80	100.00	59.66	CONECTADO
MELI	CLIENTE	12th Gen Intel(R)	2.688	16	476	Microsoft Windo	95.52	1.01	100.00	334.49	CONECTADO

```
406         specs.get("CPU Libre"),
407         specs.get("Memoria Libre"),
408         specs.get("Ancho de Banda Libre"),
409         specs.get("Disco Libre"),
410         "CONECTADO"
411     });
412 }
413
414 public boolean isConnected() {
415     return connected;
416 }
417 }
```

IA
JRE System Library [JavaSE-17]
src
 Client.java
 Client2.java
 package-info.java
 Server.java
 Server2.java
 logback.xml
Maven Dependencies
bin
target
pom.xml

```
376 // Forzar reordenamiento despues de Server2 (Java Application) C:\Program Files\Java\jdk-22\bin\javaw.exe (17 nov 2024, 11:27:15 p.m.) [pid: 45660]
377 sortByCpuFree());
378 });
379 }
380
381 private void updateRowData(int row, Map<Strin
382 // Solo actualizar si el cliente está c
383 if (model.getValueAt(row, 11).equals("C0
384     model.setValueAt(specs.get("Modelo d
385     model.setValueAt(specs.get("Velocida
386     model.setValueAt(specs.get("Nucleos"
387     model.setValueAt(specs.get("Capacida
388     model.setValueAt(specs.get("Version
389     model.setValueAt(specs.get("CPU Libr
390     model.setValueAt(specs.get("Memoria
391     model.setValueAt(specs.get("Ancho de
392     model.setValueAt(specs.get("Disco Li
393
394 }
```

Servidor

ID	Cliente/Servidor	Modelo Procesa	Velocidad (GHz)	Nucleos	Disco Duro Total	SO	CPU Libre (%)	Memoria Libre (...)	Ancho Banda Lib	Disco Libre (GB)	Estado
MELI	SERVIDOR	12th Gen Intel(R)	2.688	16	476	Microsoft Windo	98.88	1.02	100.00	334.50	CONECTADO
heidi	CLIENTE	11th Gen Intel(R)	2.688	12	476	Microsoft Windo	98.32	1.82	100.00	59.67	CONECTADO

```
406         specs.get("CPU Libre"),
407         specs.get("Memoria Libre"),
408         specs.get("Ancho de Banda Libre"),
409         specs.get("Disco Libre"),
410         "CONECTADO"
411     });
412 }
413
414 public boolean isConnected() {
415     return connected;
416 }
417 }
```

CLIENTE

----- ¡Datos Actualizandose! -----

ID: heidi

Modelo del Procesador: 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz

Velocidad del Procesador: 2.688 GHz

Nucleos: 12

Capacidad del Disco Duro: 476 GB

Version del Sistema Operativo: Microsoft Windows 11 (Home) build 22631

CPU Libre: 96.65%

Memoria Libre: 0.91 GB

Ancho de Banda Libre: 100.00%

Disco Libre: 60.05 GB

Problems @ Javadoc Declaration Console X
Client2 [Java Application] C:\Users\andym\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_

----- □Datos Actualizandose! -----

ID: andym

Modelo del Procesador: AMD Ryzen 5 3450U with Radeon Vega Mobile Gfx

Velocidad del Procesador: 2.096 GHz

Nucleos: 8

Capacidad del Disco Duro: 238 GB

Version del Sistema Operativo: Microsoft Windows 11 (Home) build 22621

CPU Libre: 93.21%

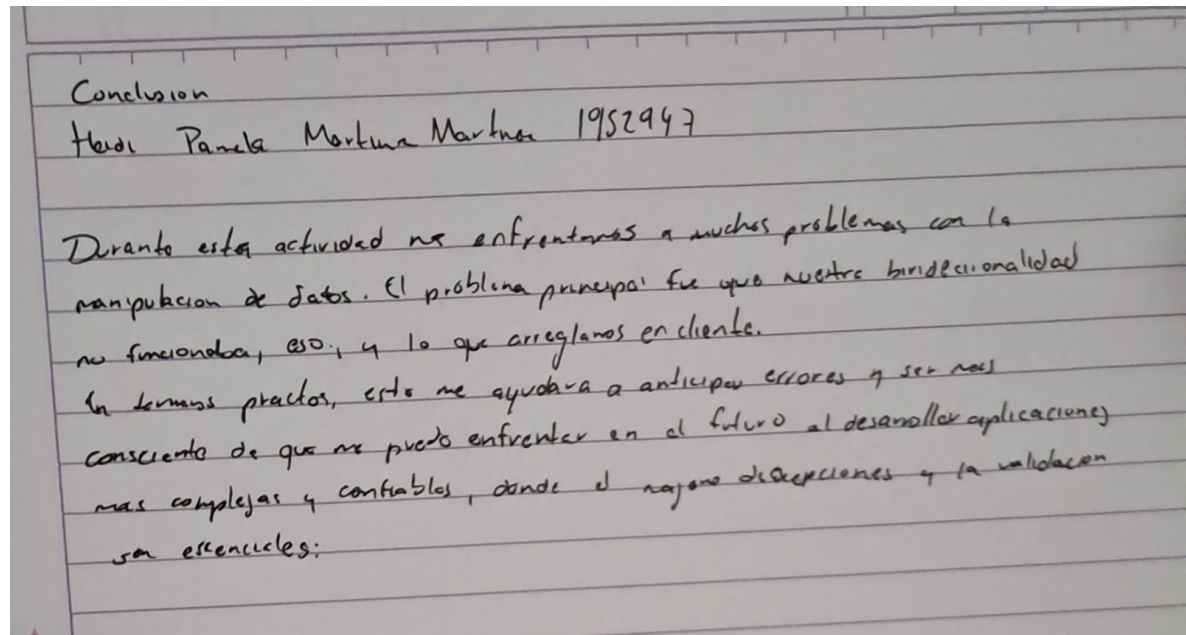
Memoria Libre: 0.56 GB

Ancho de Banda Libre: 74.00%

Disco Libre: 5.33 GB

Conclusiones

1952947 - Heidi Pamela Martínez Martínez



Andrea Monserrat Méndez Flores 1845517

Conclusión:

Esta tarea a nuestro parecer ya se un tanto más complicada, ya que obtener la información de los dispositivos y rankearlos se lograba, pero al querer intercambiar por medio de ese ranking el servidor, era lo que se estaba complicando, ya una vez logrado externar se pudo continuar, ya que básicamente era lo que se había realizado en prácticas anteriores, agregando esta nueva característica.

Conclusión Tarea Practica #7

Bidireccionalidad

Alexia Hernandez Andrade

Esta actividad permitio implementar la bidireccionalidad, lo cual fue un paso clave para mejorar la flexibilidad y eficiencia de nuestra red distribuida, al desarrollar la capacidad de cambiar dinamicamente el rol de servidor a cliente con el mejor ranking. Se logro garantizar que el dispositivo con los mejores recursos gestione la red. Es por eso que esta actividad permitio comprender la importancia de la adaptabilidad en los sistemas distribuidos y el manejo eficiente de recursos.

Sistemas Distribuidos y Paralelos

Conclusión Individual

Alejandro Sauqué Pérez

Me parece que los conceptos vistos hasta ahora en las prácticas nos permiten ver un sistema mucho más robusto cada vez. Ahora que se implementa la bidireccionalidad es un poco raro pensar como un nodo pasa a ser servidor de un momento a otro. Es notable decir que sería buena idea tener alguna característica que nos permita hacer que el sistema no este cambiando demasiado a cada rato. Es decir, como todo necesitamos de regulaciones que hagan que la bidireccionalidad mejore el rendimiento total del sistema en vez de empeorarlo.

Conclusión

2077884

Se tuvieron demasiados problemas en el transcurso de ir realizando la práctica, pues a pesar de que algunos clientes tuvieran características más altas al servidor, no cambiaban, se posicionaban al principio de la tabla pero seguían siendo clientes.

Se pensó en colocar un variable local (global) para que cuando se sugiera fuera el nuevo servidor pero no es el objetivo de la práctica.

Al final con varios intentos y modificaciones en el código se pudo lograr el resultado.